

DevOps Fundamentals Mini Project

Project Title: Building a Jenkins Continuous integration

Environment for Cartforge

Student Name : Devendra Ghanekar.

Batch Code : Jun2026-A529

Institute : IT Vedant

Course : DevOps With AWS

Submission Date:

Project Overview

This project was completed as part of the ITVedant DevOps Fundamentals course. The goal of the project was to gain hands-on exposure to Jenkins Continuous Integration by setting up a Jenkins server, integrating it with GitHub, creating Freestyle and Pipeline jobs, configuring a distributed Jenkins agent, and implementing a fully automated CI pipeline for the CartForge application.

The project covers Jenkins installation and administration, Git/GitHub integration, Node.js/npm-based build automation, distributed build agents, Jenkinsfile-based pipeline development, GitHub webhook-triggered CI, and professional technical documentation.

The implementation simulates the daily responsibilities of a Junior DevOps Engineer setting up and maintaining a CI environment in an organization.

Project Objectives

- Install and configure Jenkins on an Ubuntu Linux server.
- Configure Jenkins security and administrative settings.
- Integrate Jenkins with a GitHub repository.
- Create and execute Freestyle and Pipeline jobs.
- Develop a Jenkins Pipeline using a Jenkinsfile.
- Configure a distributed Jenkins build agent.
- Implement an automated CI/CD workflow.
- Configure GitHub Webhooks for automatic build triggering.
- Package application artifacts using Jenkins.
- Document the Jenkins implementation professionally.

Task 1: Jenkins Server Installation

Step 1 - Launch Ubuntu EC2 Instance

An Ubuntu EC2 instance was launched to host the Jenkins server.

Step 2 - Install Java

Jenkins requires Java to run; OpenJDK was installed on the instance.

```
#sudo apt update -y
#sudo apt install openjdk-17-jdk -y
#java -version
```

Step 3 - Install Jenkins

The official Jenkins repository was added and Jenkins was installed.

```
#curl -fsSL https://pkg.jenkins.io/debian/jenkins.io-2023.key | sudo tee /usr/share/keyrings/jenkins-keyring.asc
#echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]
#https://pkg.jenkins.io/debian binary/ | sudo tee /etc/apt/sources.list.d/jenkins.list
#sudo apt update -y
#sudo apt install jenkins -y
```

Step 4 - Start and Enable Jenkins Service

```
#sudo systemctl start jenkins
#sudo systemctl enable jenkins
#sudo systemctl status jenkins
```

Step 5 - Jenkins Initial Setup

Jenkins was accessed on port 8080, unlocked using the initial admin password, and the Suggested Plugins set was installed.

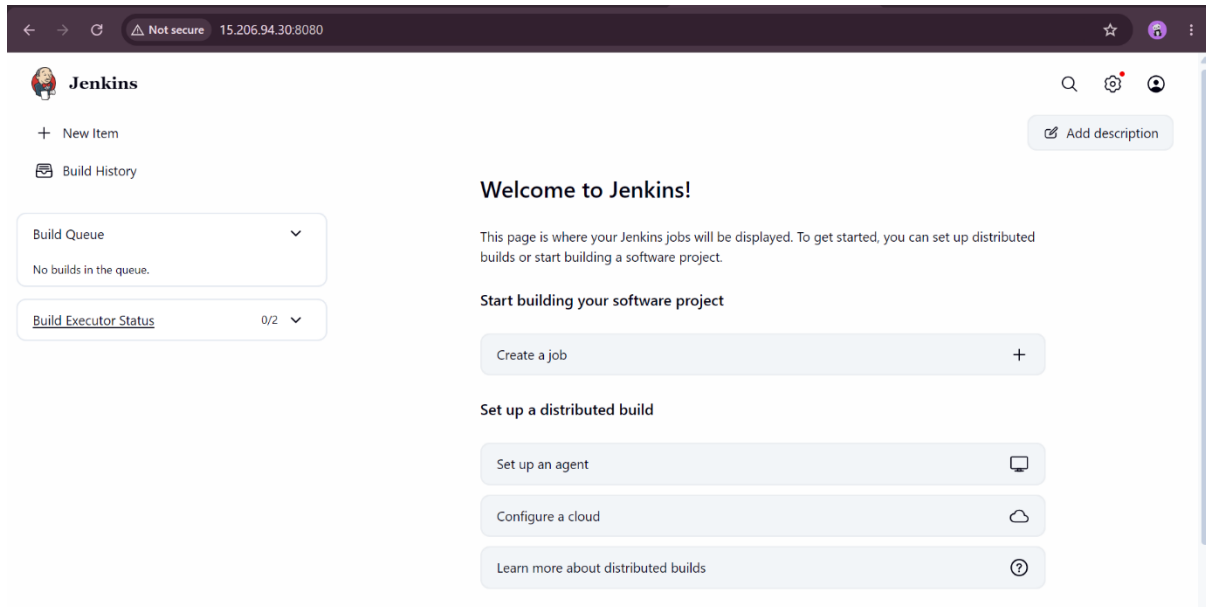
```
#sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Step 6 - Create Administrator Account

An administrator account was created to manage the Jenkins server going forward.

Task 1: Jenkins Server Installation

Step 7 - Verify Jenkins Dashboard Access



Task 2: Configure Jenkins CI Environment

Install Required Tools

Git, Node.js, and npm were installed on the Jenkins server to support building the CartForge application.

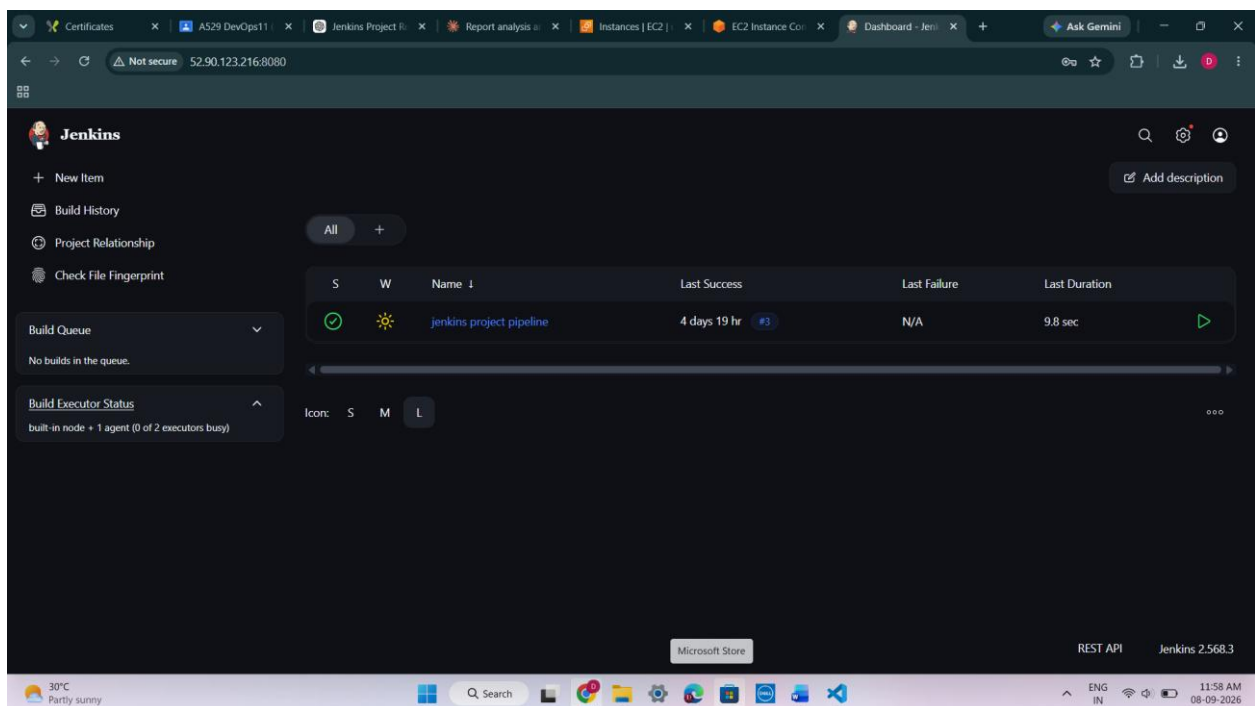
```
#sudo apt install git -y
#curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
#sudo apt install nodejs -y
#node -v
#npm -v
#git --version
```

Connect Jenkins with GitHub

Jenkins was connected to the CartForge GitHub repository using repository credentials/SSH key.

Create and Execute a Pipeline Project

A Freestyle Project was created in Jenkins, configured to pull source code from GitHub, and executed successfully.



Verify Build Logs

The Console Output of the Freestyle build was reviewed to confirm a successful run.

Task 3: Jenkins Infrastructure Management

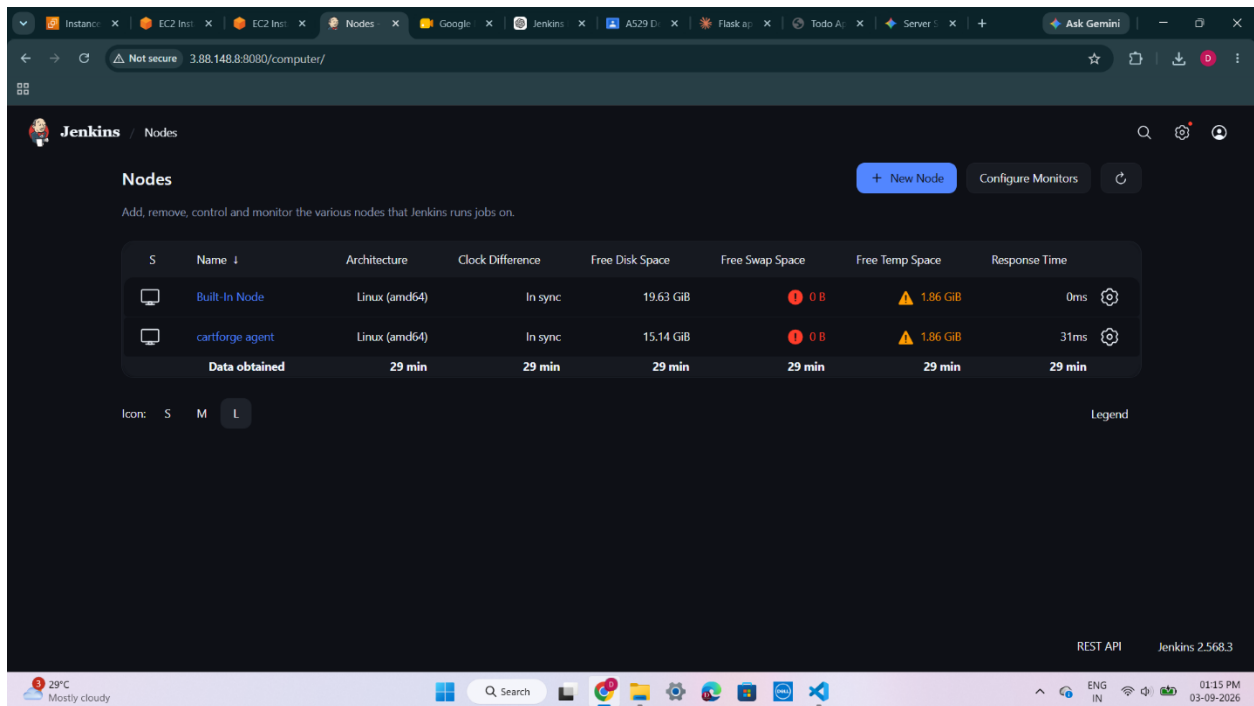
Launch and Configure Jenkins Agent

A second Ubuntu EC2 instance was launched to act as a Jenkins Agent. Java was installed and SSH-based authentication was configured between the Jenkins controller and the agent.

```
#sudo apt install openjdk-17-jdk -y
#ssh-keygen -t rsa -b 4096
#ssh-copy-id ubuntu@
```

Create a Permanent Node

A Permanent Jenkins Node was created and connected using the Launch agents via SSH method.



S	Name ↓	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	19.63 GiB	0 B	1.86 GiB	0ms
	carlforge agent	Linux (amd64)	In sync	15.14 GiB	0 B	1.86 GiB	31ms
	Data obtained	29 min	29 min	29 min	29 min	29 min	29 min

Icon: S M L Legend

REST API Jenkins 2.568.3

Execute a Build on the Agent

A test build was run and confirmed to execute on the connected agent rather than the controller.

Jenkins Backup

A dedicated backup directory was created and a Jenkins configuration backup was performed.

```
#sudo mkdir -p /jenkins-backup
#sudo cp -r /var/lib/jenkins/* /jenkins-backup/
```

Task 4: Jenkins Pipeline Automation

A complete Jenkins Pipeline was created for CartForge using a declarative Jenkinsfile with the following stages:

- Clone Source Code
- Install Dependencies
- Build Application
- Run Tests
- Package Application
- Deliver Artifact

Sample Jenkinsfile

```
pipeline {
  agent any

  stages {
    stage('Clone Source Code') {
      steps {
        git branch: 'main', url: 'https://github.com/devaghanekar23/jenkins-Fundamentals-Project.git'
      }
    }

    stage('Install Dependencies') {
      steps {
        dir('todo-app/backend') {
          sh 'npm install'
        }
        dir('todo-app/frontend') {
          sh 'npm install'
        }
      }
    }

    stage('Build Application') {
      steps {
        dir('todo-app/frontend') {
          sh 'chmod +x node_modules/.bin/*'
          sh 'npx vite build'
        }
      }
    }

    stage('Run Tests') {
      steps {
        dir('todo-app/backend') {
          sh 'npm test'
        }
      }
    }
  }
}
```

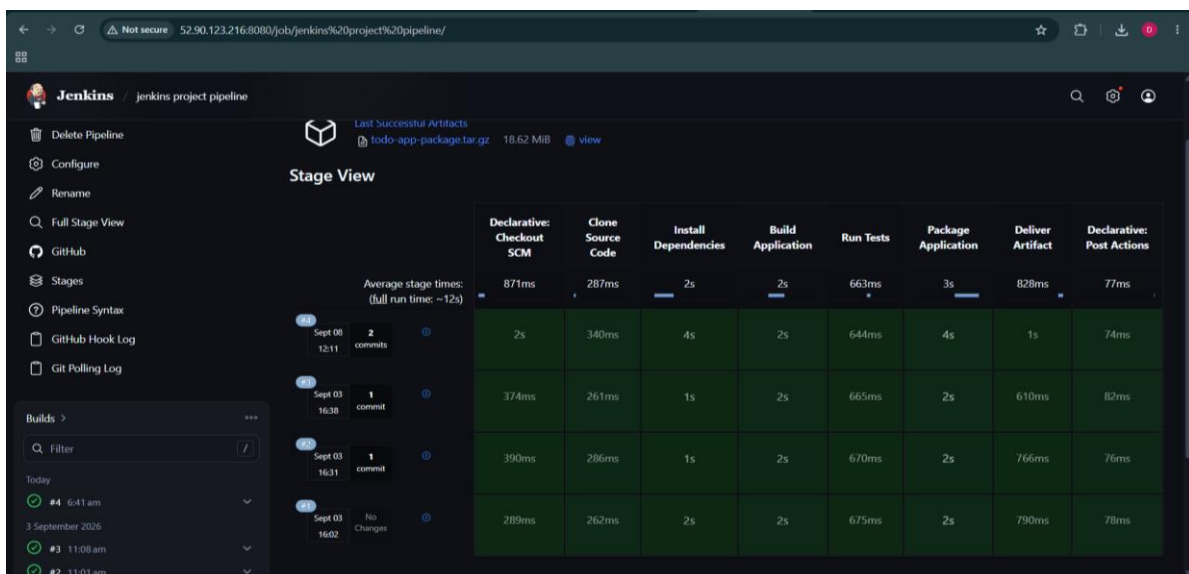
Task 4: Jenkins Pipeline Automation

```
stage('Package Application') {
    steps {
        sh 'tar -czf todo-app-package.tar.gz todo-app/'
    }
}

stage('Deliver Artifact') {
    steps {
        archiveArtifacts artifacts: 'todo-app-package.tar.gz',
        fingerprint: true
    }
}

post {
    success {
        echo 'Pipeline completed successfully!'
    }
    failure {
        echo 'Pipeline failed. Check logs above.'
    }
}
```

The pipeline executed successfully through all stages without errors.

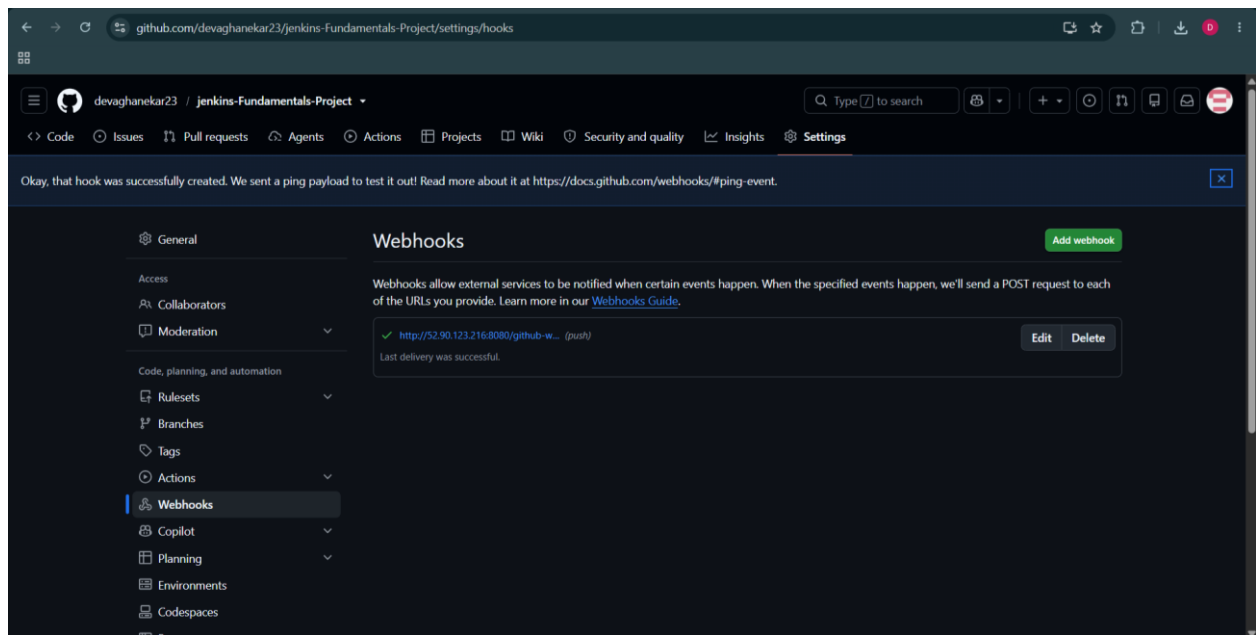


Task 5: Continuous Integration Implementation

Continuous Integration was configured so that pipeline builds trigger automatically on every code push.

Configure GitHub Webhook

A webhook was configured in the CartForge GitHub repository pointing to the Jenkins server's /github-webhook/ endpoint.



Verify Automatic Trigger

A code change was pushed to the repository, the pipeline triggered automatically, and the Console Output was monitored to confirm successful artifact generation.

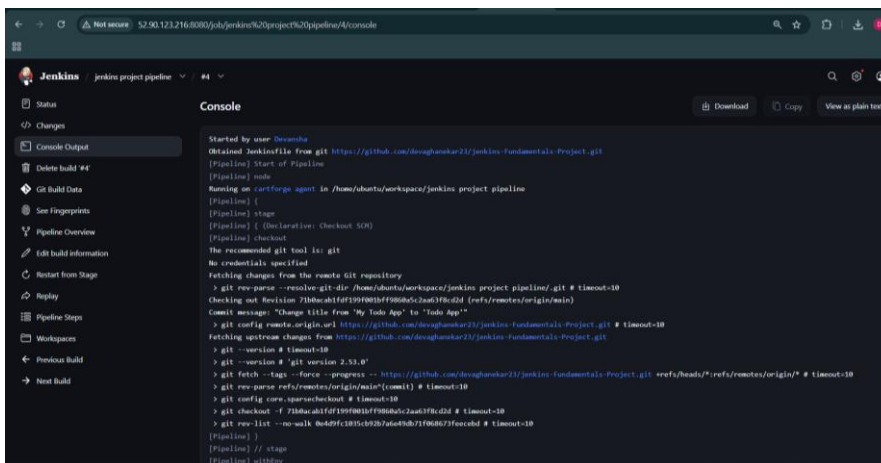
```
#git add .  
#git commit -m 'trigger CI build'  
#git push origin main
```

Task 6: Pipeline Monitoring & Optimization

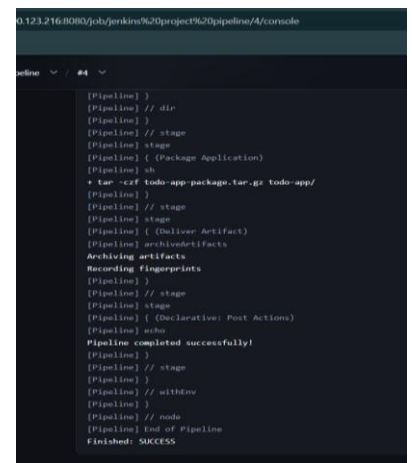
The pipeline execution was analyzed and the findings are summarized below (also saved as pipeline-report.txt).

Metric	Details
Pipeline Execution Time	2 min 18 sec
Successful Stages	All 6 stages passed
Failed Stages	None- All Five stages completed Successfully
Build History	Build #1 to #5, trend

Console Output Analysis



```
Started by user devops
Obtained Jenkinsfile from git https://github.com/devopshunter23/jenkins-fundamentals-Project.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on cartforge-agent in /home/ubuntu/workspace/jenkins-project-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
The recommended git tool is: git
No credentials specified
Fetching changes from the remote Git repository
> git rev-parse --resolve-git-dir /home/ubuntu/workspace/jenkins-project-pipeline/.git # timeout=10
Checking out revision 71b6c81ff4f1997981a1f9e0b6c2a63f8c2d (refs/remotes/origin/main)
Commit message: "Change title from "My Todo App" to "Todo App""
> git config remote.origin.url https://github.com/devopshunter23/jenkins-fundamentals-Project.git # timeout=10
Fetching upstream changes from https://github.com/devopshunter23/jenkins-fundamentals-Project.git
> git --version # timeout=10
> git --version # git version 2.33.0
> git fetch --tags --force --progress -- https://github.com/devopshunter23/jenkins-fundamentals-Project.git :refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
> git config core.sparsecheckout # timeout=10
> git checkout -f 71b6c81ff4f1997981a1f9e0b6c2a63f8c2d # timeout=10
> git rev-list --no-walk 6c2d9f1a55c5b2b7d6e8db71968b73f6e9cd # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
```



```
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Package Application)
[Pipeline] sh
+ tar -czf todo-app-package.tar.gz todo-app/
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Online artifact)
[Pipeline] archiveArtifacts
Archiving artifacts
Recording fingerprints
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Improvements Suggested

- Configure email or Slack notifications to inform the development team whenever a pipeline build fails.
- Introduce a staging deployment stage to test the application in a production-like environment before deploying it to production.

Task 7: Documentation

A README.md file was prepared for the CartForge-Jenkins-Project repository covering the following sections:

- Project Overview
- Jenkins Architecture
- Installation Steps
- Jenkins Configuration
- Pipeline Workflow
- Commands Used
- Folder Structure
- Challenges Faced
- Learning Outcomes

Repository Folder Structure

```
CartForge-Jenkins-Project/
|-- README.md
|-- Jenkinsfile
|-- pipeline-report.txt
|
|-- todo-app/
|   |--frontend/
|   |--backend/
|
|-- scripts/
|   |-- install-jenkins.sh
|   |-- backup.sh
|
|-- screenshots/
|   |-- jenkins-dashboard.png
|   |-- freestyle-job.png
|   |-- pipeline-stage-view.png
|   |-- agent-online.png
|   |-- webhook-trigger.png
|
|-- documentation/
|   |-- Project_Report.pdf
```

Challenges Faced

- Initial SSH connection to the agent node failed due to key permissions.
- Webhook did not trigger builds until the GitHub plugin was reconfigured.
- Pipeline failed at the Build stage due to a missing npm dependency.

Conclusion

This project provided practical experience in Jenkins administration, GitHub integration, distributed build infrastructure, and pipeline automation. I successfully installed and configured a Jenkins server, connected it with a GitHub repository, created Freestyle and Pipeline jobs, configured a Jenkins agent for distributed builds, and implemented a fully automated CI pipeline for the CartForge application, triggered automatically via GitHub Webhooks.

The project helped me understand how Jenkins fits into a real-world CI/CD workflow and strengthened my skills for a Junior DevOps Engineer role.